

OPERATING SYSTEMS: MODULE 1

Architecture, Components, Dual Views & Kernel Internals

Comprehensive Hinglish Study Notes | CS_Fundamental-00 | Shivam Keshari

1. Operating System (OS) kya hai aur iska Position

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference Note: Yeh topic aapke dwara provide ki gayi **Image 1** par aadharit hai jisme OS ki definition, concentric position diagram aur computer system ke 4 components explain kiye gaye hain.

1.1 Operating System ki Basic Definition

- **Master Program:** Operating System ek aisa **program / system software** hai jo computer ke physical hardware (H/W) ko poori tarah manage aur control karta hai.
- **Intermediary / Bridge Role:** Yeh user aur raw computer hardware ke beech ek **intermediary (madhyasth)** ka kaam karta hai. Agar OS na ho, toh user ko computer ke binary circuits, registers aur memory buses se direct baat karni padegi jo aam insaan ke bas ki baat nahi hai.
- **Basis for Application Programs:** OS ek aisa basic foundation provide karta hai jiske upar saare application software (jaise Web Browser, MS Office, Code Editors, Games) install hokar run kar sakte hain.
- **User-Friendly Environment:** OS ek convenient aur easy-to-use environment taiyar karta hai jisme user bina kisi technical complexity ke apne programs execute kar sake.

1.2 Diagram Reference & Handwritten Notes (Image 1)

Original Image 1 mein ek concentric circle (onion-like) diagram diya gaya hai jiska title hai Fig: - Position of Operating System. Image 1 ke right side mein 3 items/entities note kiye gaye hain:

Fig 1.1: Position of Operating System (Concentric View)

Reference: Image 1 [Position of OS & 3 Entities Architecture]

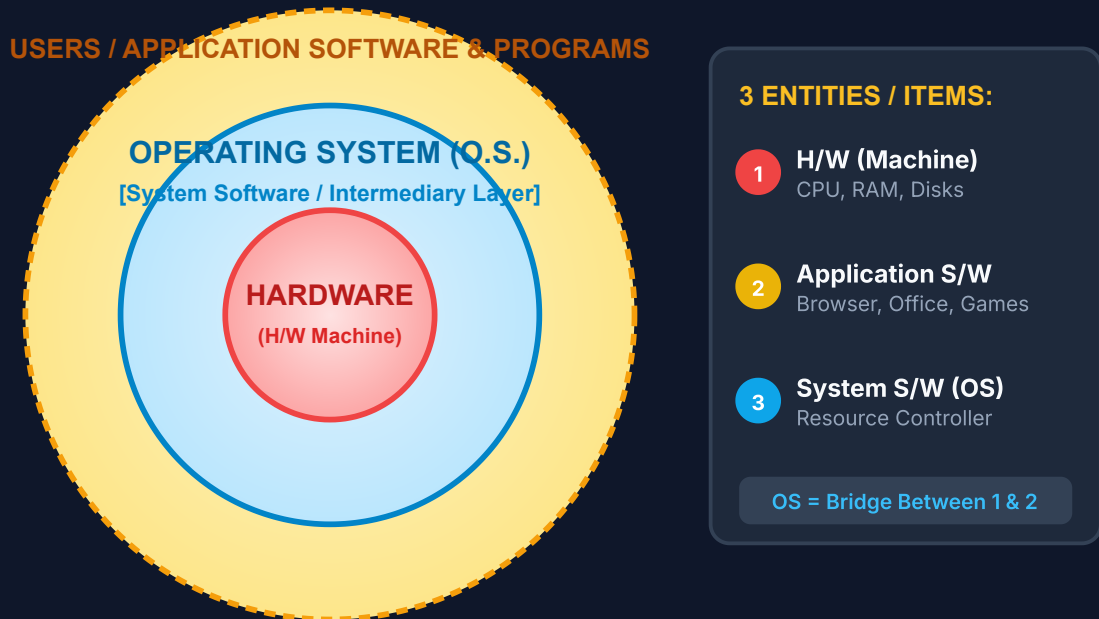


Fig 1.1: Position of Operating System & 3 Core Entities Architecture

1.3 Computer System ke 3 Main Entities (Items)

- **1.3.1 Hardware (H/W - Machine):** Physical components (CPU, RAM, Hard Disk, Motherboard, Graphics Card). Yeh computing capability aur raw processing power provide karta hai.
- **1.3.2 Application Software (Application S/W):** Specific end-user problems ko solve karne ke liye banaye gaye programs (e.g., Browser, Word Processor, Media Player, Games).
- **1.3.3 System Software (OS & Utilities):** Hardware aur application software ke beech coordination banata hai aur resource allocation control karta hai (~50% core management share).

1.4 Computer System ke 4 Core Components

Computer system ko 4 mukhya bhaagon mein divide kiya jata hai:

1. **The Hardware (H/W):** Basic computing resources provide karta hai (CPU, Memory, I/O Devices).
2. **The Operating System (O.S.):** Hardware ke use ko control aur coordinate karta hai different users aur application programs ke beech.
3. **The Application Programs:** Define karta hai ki hardware resources ka use karke user ki computing problems ko kaise solve kiya jaye.
4. **The Users:** End-users jo computer ko use karte hain (Humans, remote machines, ya automated background bots).

2. Computer System ka Abstract View aur Component Hierarchy

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concept: Jaise humne **Concept 1.4** mein dekha ki computer system ke 4 components hote hain (Hardware, OS, Application Programs, Users), yeh topic **Image 2** par aadharit hai jo in components ka layered **Abstract View** aur right side mein **Translators** ka flow explain karta hai.

2.1 Abstract View of Components (Layered Architecture)

- **Layer 1 (Bottom Most) - Computer Hardware (H/W):** Core machine jisme CPU, Main Memory (RAM), aur I/O devices aate hain.
- **Layer 2 - Operating System (OS):** Hardware ke theek upar rehta hai aur hardware ko wrap karke safe, uniform services deta hai.
- **Layer 3 - System & Application Programs:** OS ke upar different utilities run hoti hain: COMPILER, ASSEMBLER, TEXT EDITOR, DATABASES, SPREADSHEETS, GAMES.
- **Layer 4 (Top Most) - Multiple Users:** User 1, User 2, ... User N jo concurrently system use kar rahe hote hain.

2.2 Translators (Handwritten Note Reference from Image 2)

Image 2 ke top-right corner mein likha hai: TRANSLATORS :- 1) ASSEMBLER 2) COMPILER, INTERPRETER | HLL -> 0, 1.

- **Translators ki Zaroorat:** Computer Hardware (CPU) sirf aur sirf **Machine Language (0 aur 1 / Binary Bits)** samajhta hai. Hum insaan high-level English-like code likhte hain (C, C++, Java, Python). Translators ka kaam iss Human-readable code ko Computer-readable binary bits (0, 1) mein convert karna hai.
- **2.2.1 Assembler:** Assembly Language (Low-Level Language jaise MOV AX, BX) ko seedhe Machine Language (0 aur 1) mein convert karta hai.
- **2.2.2 Compiler:** High-Level Language (HLL jaise C/C++) ke poore code ko ek saath compile karke machine code generate karta hai.
- **2.2.3 Interpreter:** High-Level Language (jaise Python, JS) ko line-by-line interpret karke execute karta hai.

Fig 2.1: Abstract View of Computer System Components

Reference: Image 2 [Layered Components & Translators Architecture]

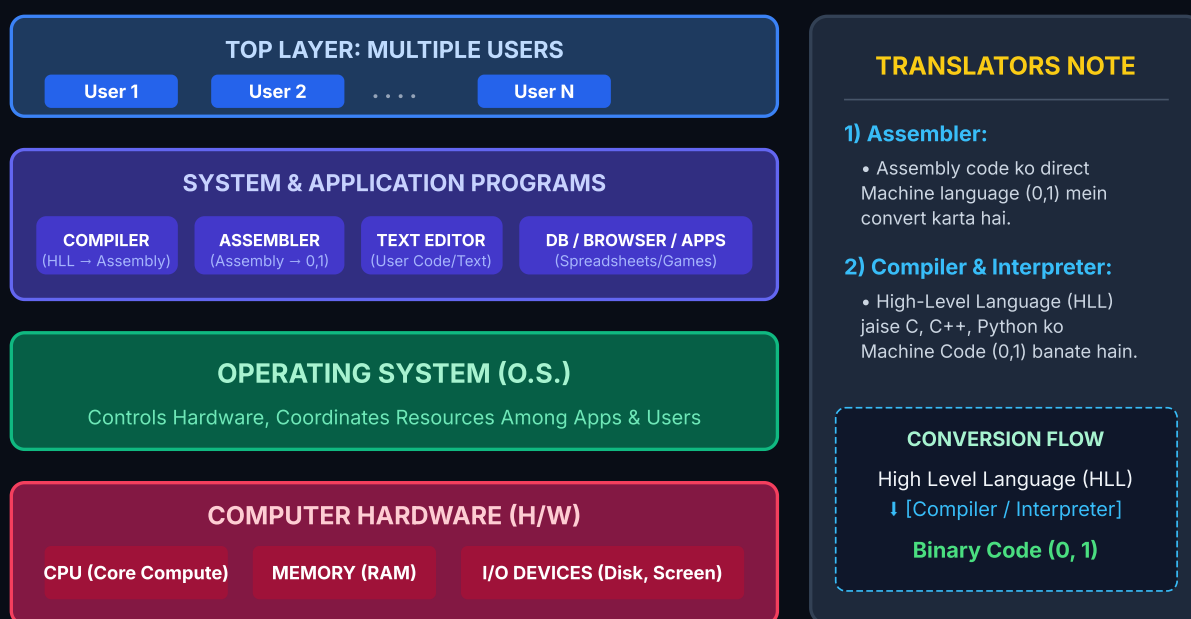


Fig 2.1: Abstract View of Computer System & Translators Flow

2.3 Hardware Resources (H/W Deep Dive)

- **(a) The Memory (RAM):** Programs ko execute hone ke liye memory chahiye hoti hai. Variables aur active data yahin store hota hai.
- **(b) Input & Output (I/O) Devices:** Disks (Storage), Keyboard, Mouse, Monitor, Network cards jinke zariye data in aur out hota hai.
- **(c) CPU (Central Processing Unit):** Basic computing resources aur calculations execute karne wala brain.

2.4 Application Programs vs OS Coordination

Application programs (Word, Spreadsheets, Games, DBMS) solve user computing problems. OS in programs ke beech hardware resources ka coordination karta hai taaki koi application hardware ko monopolize na kare aur system stable bana rahe.

3. User's aur System's Point of View of Operating System

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concepts: Concept 1.2 mein OS ko intermediary aur Concept 2.1 mein Abstract View dekha gaya tha. Yeh topic **Image 3** par aadharit hai jisme OS ko do mukhya perspectives se dekha jata hai: **User View** aur **System View (Resource Manager)**.

3.1 User View (End-User Perspective)

- **Primary Goal: Convenience, Usability & Responsiveness:** User ke liye OS ka matlab hai aasan aur bina rukawat computer chalana.
- **Interface Modes:** GUI (Windows, icons), CLI (Terminal commands), aur APIs (Developers ke liye).
- **User Focus:** Running applications, managing files, configuring settings, and interacting with peripherals.

3.2 System View (Resource Manager Perspective)

Image 3 handwritten note: "**Resource Manager**" (marked very important).

- **Primary Goal: Efficiency, Control & Resource Allocation:** System perspective se OS ek Resource Allocator hai. Hardware limited hai (CPU cores, RAM, I/O bandwidth). OS ka kaam hai fair aur collision-free distribution karna.
- **Core Components:** Kernel, Device Drivers, Memory Management, Process Management, File Systems, Networking Protocols.
- **Emphasis:** Efficiency, fair resource allocation, security aur stability.

Fig 3.1: Dual Perspectives of Operating System

Reference: Image 3 [User View vs System View / Resource Manager]

1. USER VIEW (End-User)

Primary Goal: Convenience & Usability

- Platform for executing applications
- Interfaces: GUI, CLI, APIs
- Focus: Running apps, files, settings
- User concerns: Responsiveness, ease of use

Example: User Perspective

User writes code:

```
a = b + c;
```

User only cares about output correctness and fast execution!

2. SYSTEM VIEW (Resource Manager)

Primary Goal: Efficiency & Control

- OS as a master Resource Allocator
- Controls CPU, Memory, I/O devices
- Enforces Security, Stability & Isolation
- Target audience: SysAdmins, Engineers

Example: System Perspective

Inside OS & Hardware:

1. Memory allocated for a, b, c
2. Values loaded in registers (R1, R2)
3. CPU executes ADD, result written to RAM
4. Protection prevents illegal memory access

3.3 Numerical / Computational Example & Step-by-Step Thought Process: $a = b + c$

Image 3 handwritten note: $a = b + c$. Aaiye iska complete thought process samjhein:

Scenario: User writes `int b = 10; int c = 20; int a = b + c;`

1. **User View Thought Process:** User chahta hai ki `10 + 20` add hokar turant `30` mil jaye. User ko underlying hardware address se matlab nahi hai.

2. **Translators Role (Ref Concept 2.2):** Compiler ne iss instruction ko machine code mein toda:

- `LOAD R1, [Addr_B]` (Value 10 in Register R1)
- `LOAD R2, [Addr_C]` (Value 20 in Register R2)
- `ADD R3, R1, R2` (ALU performs addition: 30)
- `STORE [Addr_A], R3` (Value 30 stored in RAM address A)

3. **System View (Resource Manager) Thought Process:**

- *Process Allocation:* OS iss code ko PID deta hai aur isolated RAM allot karta hai.
- *Memory Protection:* MMU check karta hai ki process apni boundary ke bahar na jaye.
- *CPU Scheduling:* OS process ko CPU cycle allot karta hai aur context switch ke samay register state save karta hai.
- *I/O Signaling:* Final output screen driver ke zariye user ko deliver hota hai.

3.4 User View vs System View Comparison Table

Feature	User View (End-User)	System View (Resource Manager)
Main Objective	Convenience & Usability	Efficiency & Resource Optimization
Interaction Mode	GUI, CLI, Desktop Apps	System Calls, Interrupts, Drivers
Focus Area	App execution & output response	Memory Allocation, CPU Scheduling, Security
Perspective on $a=b+c$	"Sirf 30 answer aana chahiye."	"RAM allocate hui, Registers load hue, MMU verified!"

4. Kernel aur uske Core Operations (With Types of Kernel)

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concepts: Concept 1.1 mein humne dekha tha ki OS hardware manage karta hai aur Concept 3.2 (System View) mein Kernel ko core component bataya gaya tha. Yeh topic **Image 4** par aadharit hai jo Kernel ke **5 Fundamental Operations** aur **3 Types of Kernel** ko detail karta hai.

4.1 Kernel kya hota hai (The Core Bridge)

- **Heart of the OS:** Kernel OS ka core program hai jo boot hote hi RAM mein load hota hai aur system chalne tak active rehta hai.
- **Bridge Role:** Yeh hardware aur software ke beech direct bridge hai. Applications hardware ko direct touch nahi kar sakte; unhe Kernel se **System Calls** ke through permission leni padti hai.

4.2 Key Functions / Operations of Kernel (5 Pillars)

1. **Process Management:** CPU time allot karna (CPU Scheduling), process creation, coordination aur termination ensure karna.
2. **Memory Management:** RAM allocation/deallocation, Memory Protection (isolation), aur Virtual Memory handle karna.
3. **Device Management:** Hardware peripherals ke device drivers handle karna aur uniform I/O interface provide karna.
4. **File System Management:** Storage devices (SSD/HDD) par files create, read, write aur organize karna.
5. **Security & Access Control:** User permissions verify karna, unauthorized access rokna aur system integrity maintain karna.

Fig 4.1: Kernel Architecture & Core Operations

Reference: Image 4 [Kernel Functions & 3 Types Comparison]

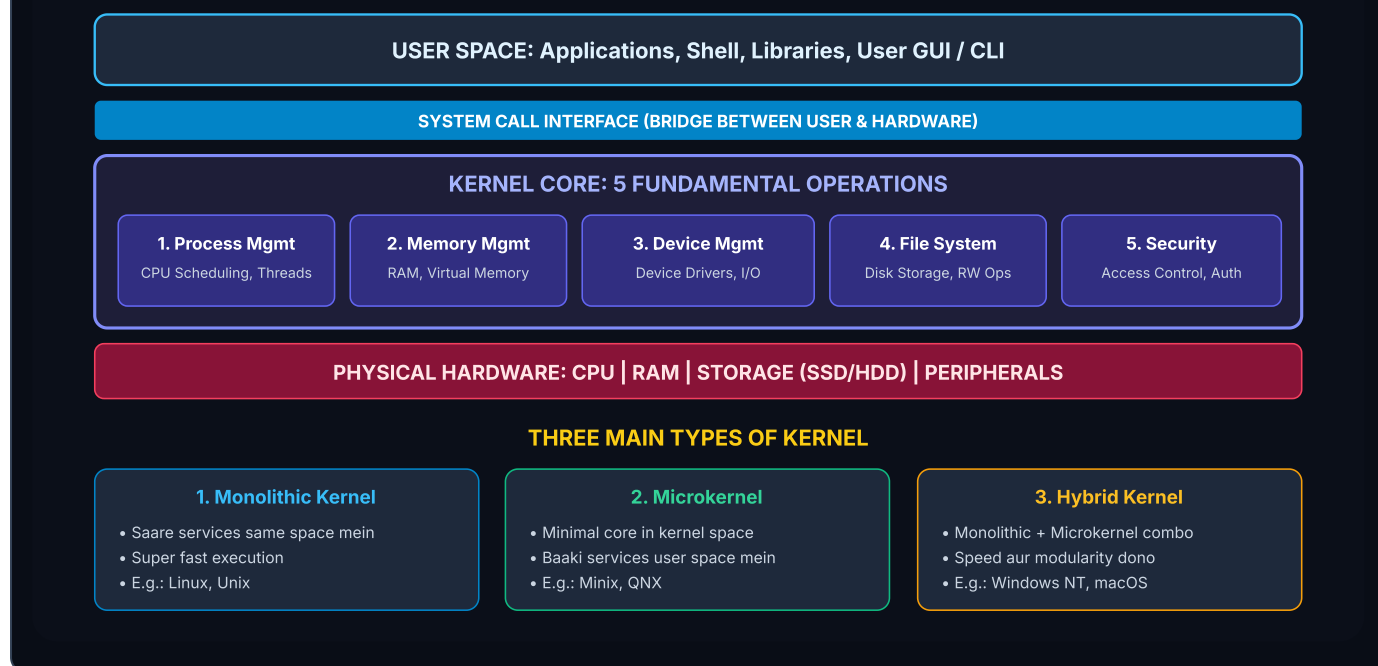


Fig 4.1: Kernel Architecture, 5 Operations & 3 Kernel Types

4.3 Types of Kernel (Image 4 Reference)

4.3.1 Monolithic Kernel

- Definition:** Saare OS services (Process, Memory, File System, Drivers) **ek hi address space (Kernel Space)** mein run karte hain.
- Pros:** Super fast execution (direct memory function calls).
- Cons:** Agar ek driver crash ho, toh poora OS crash ho sakta hai.
- Examples:** Linux, Traditional Unix.

4.3.2 Microkernel

- Definition:** Kernel space ko **bahut chhota (minimal)** rakhta hai. Baaki services User Space mein servers ke roop mein chalti hain.
- Pros:** Highly secure aur crash-resistant (Driver crash hone se OS nahi marta).
- Cons:** IPC (message passing) overhead ki wajah se thoda slow.
- Examples:** Minix, QNX, L4.

4.3.3 Hybrid Kernel

- Definition:** Monolithic ki high speed aur Microkernel ki modularity ko combine karta hai.
- Examples:** Windows NT (Windows 10/11), macOS (XNU).

4.4 Kernel Types Comparison Table

Paimana	Monolithic Kernel	Microkernel	Hybrid Kernel
---------	-------------------	-------------	---------------

Address Space	Saare services Kernel Space mein	Sirf core kernel space mein, baaki User Space	Mixture (Critical parts in kernel space)
Speed	Bahut Fast	Thoda Slow (IPC overhead)	Fast & Balanced
Fault Tolerance	Low (Driver crash = OS crash)	Very High (Stable)	High
Real Examples	Linux, Unix	Minix, QNX	Windows NT, macOS